

 ESPE UNIVERSIDAD DE LAS FUERZAS ARMADAS INNOVACIÓN PARA LA EXCELENCIA	GUIA PARA LAS PRÁCTICAS DE LABORATORIO, TALLER O CAMPO	Departamento de Ciencias de la Computación
--	---	---

DEPARTAMENTO:	CIENCIAS DE LA COMPUTACIÓN		CARRERA:	INGENIERÍA EN TECNOLOGÍAS DE LA INFORMACIÓN	
ASIGNATURA:	Programación Avanzada		PERÍODO LECTIVO:	202551	NIVEL: 7mo
DOCENTE:	Ing. Luis Castillo, Mgtr.		NRC:	29100	PRÁCTICA N°: 5
LABORATORIO DONDE SE DESARROLLARÁ LA PRÁCTICA			Laboratorio G-206		
TEMA DE LA PRÁCTICA:	Pruebas Unitarias				
INTRODUCCIÓN:					
En este laboratorio se implementan pruebas unitarias para clases de servicio en un proyecto Spring Boot. Se aplicará el patrón AAA (Arrange–Act–Assert), se simularán dependencias con Mockito y se verificará tanto el resultado como las interacciones con repositorios y servicios externos. El propósito es garantizar la corrección del comportamiento del negocio de forma aislada, rápida y repetible.					
OBJETIVOS:					
• Diseñar y ejecutar pruebas unitarias que cubran escenarios positivos, negativos y casos borde de un servicio. • Simular dependencias (Repository/Client) con Mockito y verificar interacciones con verify, when, thenReturn y ArgumentCaptor. • Evaluar calidad mínima de pruebas aplicando buenas prácticas: legibilidad AAA, independencia, aserciones claras y cobertura funcional.					
MATERIALES:					
REACTIVOS: No aplica			INSUMOS: • Una PC con Windows/Linux • IntelliJ IDEA Ultimate • Acceso a Internet		
EQUIPOS:					
Windows 10 o superior, Procesador Intel® Core™ i7-6700T o superior, 12GB RAM o superior, 480GB SSD o superior, Intel HD Graphics 530, similar o superior.					
MUESTRA: No aplica					
INSTRUCCIONES:					
1. Utilizar como material principal de apoyo, aquel indicado en clase por el docente. 2. No olvide incluir capturas de pantallas de todas las actividades realizadas durante la práctica. 3. En los datos ingresados, por favor usar sus datos personales, con el fin de verificar la realización de este trabajo. 4. Se debe comentar el código como mejor práctica de programación.					
ACTIVIDADES POR DESARROLLAR:					
PARTE 1: Creación del proyecto base					
Paso 1: Crear un nuevo proyecto de SpringBoot en IntelliJ					
a. Escoger bien la configuración del proyecto especialmente la versión de java. b. No es necesario añadir dependencias ya que el proyecto que se crea ya tiene la dependencia <i>spring-boot-starter-test</i> (verificar que esté incluido en el archivo <i>build.gradle</i>).					

Paso 2: Crear la estructura del proyecto.

- a. Crear las siguientes carpetas, dentro del src:

```
src/main/java/...  
    service/  
    repository/  
    model/  
    dto/  
src/test/java/...  
    service/
```

PARTE 2: Implementación mínima**Paso 1: Crear Modelo.**

- a. Un pedido tiene: customerEmail, amount, status.
b. Crear getters y setter solo de status

Paso 2: Crear DTO de confirmación.

- a. En este DTO de confirmación se crearán dos funciones que permitan:
- Obtener el Id de la orden
 - Obtener un código de confirmación

PARTE 3: Dependencias a mockear**Paso 1: Implementar la dependencia OrderRepository.**

- a. Implementar una interfaz con dos métodos:
- Para guardar una orden
 - Para conocer si existe un cliente con un correo electrónico específico

Paso 2: Implementar la dependencia FraudClient.

- a. Implementar una interfaz con un método para generar un código de confirmación.

Paso 3: Implementar la dependencia ConfirmationService.

- a. Implementar una interfaz con un método para saber si un cliente ha sido bloqueado por fraude.

Paso 4: Implementar el servicio OrderService.

- a. Crear objetos pertenecientes a las dependencias e inicializarlas desde el constructor.

- b. Implementar la función para crear una orden con la siguiente lógica:
 - email debe contener @
 - amount > 0
- c. Rechazar al cliente si el cliente está “bloqueado”, llamar al servicio externo
- d. Guardar en repositorio
- e. Generar un código de confirmación llamando al servicio externo
- f. Retornar un DTO con confirmación

PARTE 4: Pruebas unitarias (JUnit 5 + Mockito, AAA)

Paso 1: Crear la clase de prueba.

- a. Manejo del decorador @BeforeEach.
- b. Crear los siguientes casos de prueba:
 - Crear orden, validar datos y retornar la confirmación
 - Crear orden con email no válido para lanzar una excepción
 - Crear orden con monto menos o igual a cero para lanzar una excepción
 - Crear orden con un usuario bloqueado debe lanzar excepción y no guardarse
 - Crear una orden con datos validos que debe guardar la misma, usando *ArgumentCaptor*

SECCIÓN DE PREGUNTAS/ACTIVIDADES

Actividad 1 — Nuevos casos de prueba negativos

Agregar dos nuevas pruebas unitarias a la clase *OrderServiceTest*, aplicando el patrón AAA, para los siguientes escenarios:

1. Email nulo
 - Entrada: email = null, amount > 0
 - Comportamiento esperado:
 - Se debe lanzar *IllegalArgumentException*
 - No debe ejecutarse ninguna dependencia (*FraudClient*, *OrderRepository*, *ConfirmationService*)
2. Monto negativo
 - Entrada: email válido, amount < 0
 - Comportamiento esperado:
 - Se debe lanzar *IllegalArgumentException*
 - No debe ejecutarse ninguna dependencia

Actividad 2 — Verificación del orden de ejecución (Mockito InOrder)

Agregar una prueba unitaria adicional que valide que, cuando el pedido es válido:

1. Primero se consulta el servicio antifraude (*fraudClient.isBlocked*)
2. Luego se guarda el pedido (*orderRepository.save*)
3. Finalmente se genera el código de confirmación (*confirmationService.generateConfirmationCode*)

RESULTADOS OBTENIDOS:

- a. Realizar el informe en el formato general de informes de laboratorio.
- b. Evidencia con capturas las actividades prácticas realizadas.
- c. Anexar el código fuente en el informe.

CONCLUSIONES:

- Escribir al menos dos conclusiones.

RECOMENDACIONES:

- Escribir al menos dos recomendaciones.

FIRMAS

F:

**Nombre: Ing. Luis Castillo, Mgtr.
DOCENTE**

F:

**Nombre: Ing. Juan Fernando Galarraga, Mgtr.
COORDINADOR DE ÁREA DE CONOCIMIENTO**

F:

**Nombre: Crnl (SP) Fidel Castro de la
Cruz
JEFE DE LABORATORIO**